ANALYSIS REPORT

Residual Harmonic Network Model Explanatory Report

Feedforward Transmission Error Baseline

Styled PDF edition generated from the canonical Markdown analysis report for improved readability, section hierarchy, and table presentation.

Overview

This report documents the implemented `residual_harmonic_mlp` model, whose Python class is `ResidualHarmonicNetwork`.

This is the most explicitly hybrid structured-neural model currently implemented in the repository.

Its design is based on a decomposition:

- one branch models the structured harmonic part;
- one branch models the residual correction;
- the final prediction is their sum.

Model Description

`ResidualHarmonicNetwork` combines

- a `HarmonicRegression` structured branch;
- a `FeedForwardNetwork` residual branch.

The structured branch aims to capture the dominant periodic TE component. The residual branch aims to capture what the harmonic model cannot explain. This is not just feature engineering. It is an explicit additive decomposition model.

Operating Principle

The model assumes that TE can be decomposed approximately as:

```
TE = structured_harmonic_component + residual_component
```

Operationally

1. the structured branch computes a harmonic prediction;
2. the residual branch computes an MLP correction;
3. the two outputs are added.

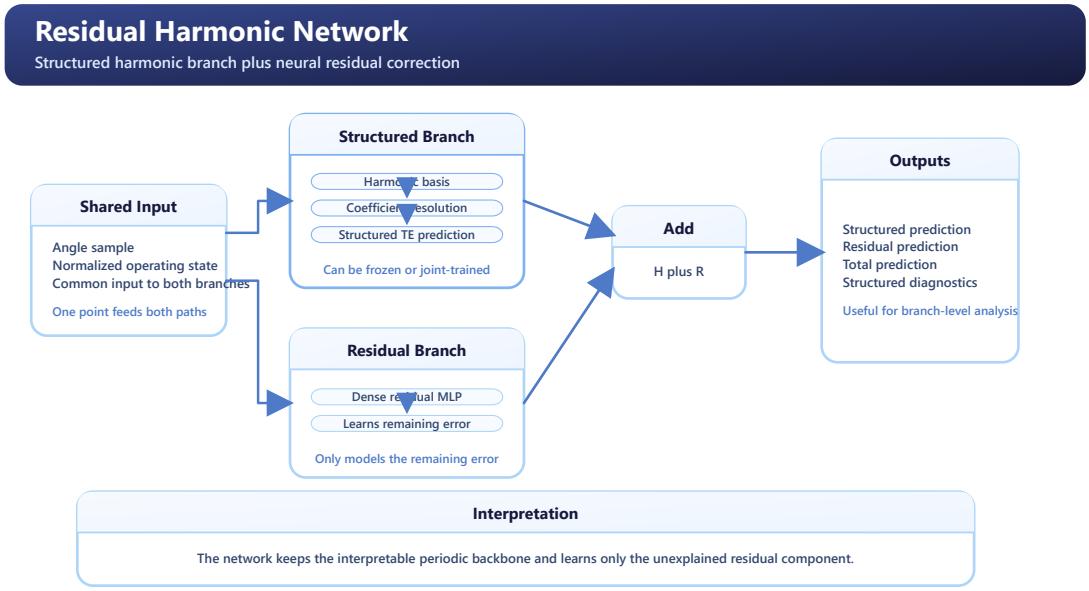
The model also supports a `freeze_structured_branch` option:

- when `True`, the harmonic branch is kept fixed and only the residual branch learns;
- when `False`, both branches are trained jointly.

This creates two useful regimes:

- fixed analytical baseline + learned correction;
- jointly optimized structured + residual hybrid model.

Conceptual Map



Residual harmonic network conceptual diagram

The model can be summarized as

```text
Input Point -> Structured Branch -> harmonic basis -> harmonic coefficients -> structured TE -> Residual Branch -> normalized input features -> MLP -> residual TE -> sum -> final TE prediction
```

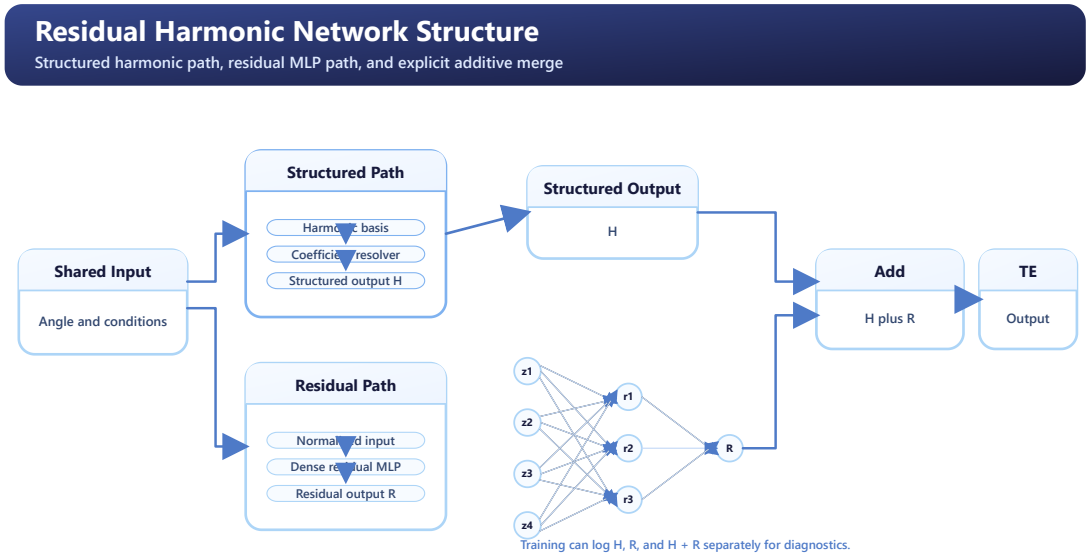
More compactly

```text
TE(theta, c) = H(theta, c) + R(theta, c)
```

where

- `H` is the structured harmonic branch;
- `R` is the learned residual neural branch.

Architecture Diagram



Residual harmonic network architecture diagram

This second diagram exposes the actual hybrid structure more concretely:

- one structured harmonic path computes `H`;
- one dense neural path computes `R`;
- the two paths are merged additively;
- the training code can still inspect the structured branch separately through auxiliary outputs.

Why This Model Exists

This model exists because the TE problem often has both:

- a clear structured periodic component;
- a remaining nonlinear residual component.

  A pure harmonic model may be too rigid. A plain MLP may hide the structure instead of exposing it. This hybrid model tries to keep both:

- structure;
- flexibility.

Advantages

- Explicit decomposition into structured and residual parts.
- Better interpretability than a pure neural network.
- More expressive than a pure harmonic regressor.
- Supports frozen or joint training strategies.
- Very aligned with the broader program direction toward physics-informed and hybrid models.

Disadvantages

- More complex than the other static baselines.
- Requires reasoning about two branches instead of one.
- Can suffer from branch overlap if the residual branch learns what the structured branch should have represented.
- Slightly harder to interpret than a pure harmonic model because total behavior is split across two modules.

Expected Behavior In The TE Context

This model is especially promising when

- periodic structure is real and important;
- but it does not explain the full TE behavior;
- nonlinear condition effects remain after the harmonic approximation.

It is a strong bridge between:

- purely structured models;
- fully flexible neural models;
- future physics-informed hybrids.

Python Model Implementation

The implementation lives in `scripts/models/residual_harmonic_network.py`.

`ResidualHarmonicNetwork.__init__(...)`

This constructor assembles the two-branch architecture.

Main responsibilities

- initialize `self.structured_branch` as `HarmonicRegression`;
- initialize `self.residual_branch` as `FeedForwardNetwork`;
- optionally freeze the structured branch parameters.

This constructor is the place where the conceptual decomposition becomes a concrete implementation.

`forward_with_input_context(...)`

This function performs the main hybrid forward pass.

It

- calls the structured branch using raw angle plus normalized conditions;
- calls the residual branch on normalized inputs;
- returns the sum.

This is the actual predictive equation of the model.

`compute_auxiliary_output_dictionary(...)`

This function is particularly important in the repository.

It returns

- `structured_prediction_tensor`
- `residual_prediction_tensor`
- `prediction_tensor`

  This makes the model more inspectable than the other baselines, because the training module can keep track of the structured component separately from the total prediction.

Repository Integration

The model is registered in `scripts/models/model_factory.py` under `model_type == "residual_harmonic_mlp"`.

Its YAML configuration controls

- harmonic order;
- coefficient mode;
- residual hidden layers;
- residual activation and dropout;
- residual layer normalization;
- whether the structured branch is frozen.

Training Workflow Overview

This model also uses the shared structured-neural training path in `scripts/training/train_feedforward_network.py`.

At high level

1. the model is instantiated from YAML;
2. the regression module detects that it provides `compute_auxiliary_output_dictionary(...)`;
3. the total prediction is used for optimization;
4. the structured branch diagnostics are also exposed during training;
5. checkpointing and testing follow the normal shared path.

Training Logic Relevant To This Model

`TransmissionErrorRegressionModule.forward_regression_model(...)`

This is the main integration point.

Because `ResidualHarmonicNetwork` exposes `compute_auxiliary_output_dictionary(...)`, the regression module:

- receives the total prediction tensor;
- keeps the auxiliary branch outputs;
- returns them inside the batch-output dictionary.

  This is more advanced than the plain contextual forward used by `harmonic_regression` and `periodic_mlp`.

`TransmissionErrorRegressionModule.compute_batch_outputs(...)`

This function merges the auxiliary outputs returned by the model into the common batch-output dictionary.

That means the training loop has access not only to:

- total prediction;

  but also to:
- structured branch prediction;
- residual branch prediction.

`TransmissionErrorRegressionModule.compute_loss(...)`

This function adds structured diagnostics when `structured_prediction_tensor` is available.

Specifically, it logs

- `structured_mae`
- `structured_rmse`

  after denormalizing the structured branch output. This is important because it lets us inspect:
- how good the harmonic branch is on its own;
- how much the residual branch is contributing.

`train_feedforward_network(...)`

The outer training flow remains shared, but this model benefits more than the others from the reusable training design because:

- the hybrid branch logic lives inside the model;
- the diagnostic logging lives inside the generic regression module;
- the orchestration code does not need to become model-specific.

  This is a good example of clean separation between:
- model-specific behavior;
- training infrastructure.

Practical Interpretation

`ResidualHarmonicNetwork` is currently the most strategically interesting static hybrid model in the repository.

It is the first implemented architecture that says, explicitly:

- keep the interpretable structured TE component;
- learn only what remains unexplained.

  That makes it a natural stepping stone toward the later hybrid and PINN-oriented stages of the program.